AutoCloud-Agent: Multi-Agent Reinforcement Learning for Cloud Resource Management

K Chaitanya (25CS60R28) Rahul K (25CS60R16) Sri Lalitha Rama Siddesh Devarakonda (25CS60R80)
Department of Computer Science and Engineering
Indian Institute of Technology Kharagpur
April 2026

***Abstract*—Cloud autoscaling requires balancing two competing objectives: *elasticity* (responding to workload changes quickly enough to meet SLAs) and *cost minimization* (not running more VMs than necessary). We present AutoCloud-Agent, a multi-agent reinforcement learning system that decomposes cloud resource management into three concurrent sub-problems—horizontal scaling, VM consolidation, and job scheduling—each handled by an independent PPO agent. A Transformer-based workload forecaster with Monte Carlo Dropout provides uncertainty-aware demand predictions, enabling proactive scaling. A rule-based Safety Coordinator enforces hard constraints (minimum node floor, boot protection, uncertainty-driven suppression) to guarantee system stability. Evaluated on the Alibaba Cluster Trace 2018, AutoCloud-Agent achieves 96.2% cost efficiency and 55.2% CPU utilization with 100% SLA compliance, matching the best classical method (MPC) while outperforming Kubernetes HPA by 3.4% on cost and 77% on CPU utilization.**

***Index Terms*—Cloud computing, autoscaling, reinforcement learning, multi-agent systems, resource management, elasticity, cost optimization**

I. Introduction

Cloud computing providers operate large-scale clusters of virtual machines (VMs) to serve diverse workloads. The fundamental challenge is deciding, at every point in time, how many VMs should be active, which idle VMs should be shut down, and how incoming jobs should be assigned to available resources—all while minimizing operational cost and meeting service level agreements (SLAs).

This problem is characterized by two competing objectives:

1) **Elasticity:** The system must scale resources up quickly when demand spikes to prevent SLA violations (e.g., P95 latency exceeding 500ms).
2) **Cost Minimization:** The system must scale down aggressively during low-demand periods to avoid paying for idle resources.

Traditional autoscaling systems such as Kubernetes Horizontal Pod Autoscaler (HPA) [8] and AWS Auto Scaling use reactive threshold-based rules (e.g., "if CPU $>$ 80%, add a node"). These approaches have two limitations: (1) they respond *after* demand increases, incurring latency during the VM boot window (typically 30–90 seconds), and (2) they treat scaling, consolidation, and scheduling as independent problems, missing coordination opportunities.

Recent work has applied reinforcement learning (RL) to autoscaling [4], [5], demonstrating that learned policies can outperform hand-tuned rules. However, most RL approaches use a single monolithic agent that must control all resource management decisions simultaneously, leading to a combinatorially large action space.

We propose **AutoCloud-Agent**, which addresses these limitations through three key contributions:

1) **Multi-agent decomposition:** Three independent PPO agents handle scaling, consolidation, and scheduling respectively, reducing the joint action space from $\sim$15 million to three manageable sub-spaces.
2) **Uncertainty-aware forecasting:** A Transformer with MC Dropout predicts future demand and quantifies prediction confidence, enabling proactive scaling.
3) **Safety-constrained execution:** A rule-based coordinator enforces hard safety constraints, guaranteeing system stability regardless of agent policy quality.

II. Related Work

A. Threshold-Based Autoscaling

Kubernetes HPA [8] computes the desired number of replicas as $\lceil \text{currentReplicas} \times \frac{\text{currentCPU}}{\text{targetCPU}} \rceil$, with a 10% dead-band to prevent oscillation. AWS Target Tracking uses a similar formula with asymmetric cooldown periods (300s for scale-down, 60s for scale-up). These methods are simple and widely deployed but purely reactive—they cannot anticipate demand changes.

B. Model Predictive Control for Cloud Scaling

Zhang et al. [4] proposed AWARE, a system that uses Model Predictive Control (MPC) with exponentially weighted moving average (EWMA) forecasts to optimize scaling decisions over a 5-step lookahead horizon. AWARE demonstrated significant improvements over reactive baselines in production cloud systems at USENIX ATC 2023. Our MPC baseline implements this approach.

C. Reinforcement Learning for Resource Management

Mao et al. [5] introduced DeepRM, applying deep RL to cluster resource management. The agent learns to pack jobs onto machines by observing resource utilization patterns.

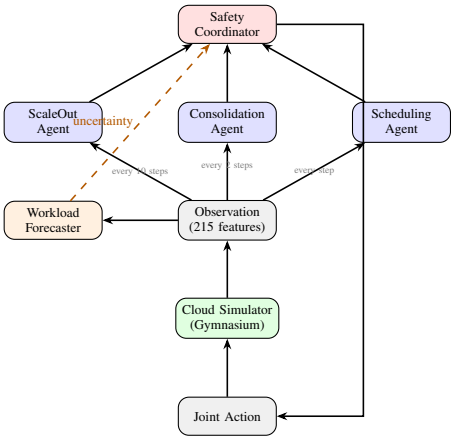

Fig. 1: AutoCloud-Agent system architecture. Three I-PPO agents operate at different temporal frequencies. The Safety Coordinator filters all actions before execution.

However, DeepRM uses a single agent with a flat action space, limiting scalability to larger clusters.

Schulman et al. [1] introduced Proximal Policy Optimization (PPO), which we use as the base RL algorithm. PPO clips the policy update ratio to prevent destructive large updates, providing stable training. We extend this to Independent PPO (I-PPO), where multiple agents share observations but maintain separate policies and reward functions.

D. Uncertainty Estimation in Neural Networks

Gal and Ghahramani [3] showed that Monte Carlo Dropout approximates Bayesian inference, providing uncertainty estimates by running multiple stochastic forward passes through a network. We apply MC Dropout to our workload forecaster, using the variance across 30 forward passes as a confidence signal that drives the Safety Coordinator's decisions.

III. System Architecture

AutoCloud-Agent consists of four components: a cloud simulator, three RL agents, a workload forecaster, and a safety coordinator. Fig. 1 shows the system architecture.

IV. Methodology

A. Problem Formulation

We model the cloud resource management problem as a multi-agent Markov Decision Process (MDP) with shared state and independent actions:

- **State** $s_t \in \mathbb{R}^{215}$: Node features (120 dims), job features (80 dims), and global features (15 dims) including forecaster predictions. All values normalized to $[0, 1]$.
- **Actions:**
  - ScaleOut: $a^{\text{so}} \in \{0, 1, 2\}$ (remove, hold, add)
  - Consolidation: $a^{\text{con}} \in \{0, 1\}^{20}$ (drain mask)
  - Scheduling: $a^{\text{sch}} \in \{0, ..., 4\}$ (priority strategy)
- **Transition:** Determined by the SimPy discrete-event simulator.
- **Rewards:** Agent-specific (described below).

B. Independent PPO (I-PPO)

Each agent is trained with Proximal Policy Optimization [1]. The clipped surrogate objective is:

$$L^{\text{CLIP}} = \mathbb{E}\left[\min\left(r_t \hat{A}_t,\ \text{clip}(r_t, 1-\epsilon, 1+\epsilon)\,\hat{A}_t\right)\right] \quad (1)$$

where $r_t = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ and $\hat{A}_t$ is the Generalized Advantage Estimate [6]:

$$\hat{A}_t = \sum_{l=0}^{T-t} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (2)$$

with $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$.

C. Temporal Hierarchy

Agents operate at different frequencies matching the natural timescale of their decisions, as shown in Table I.

TABLE I: Temporal hierarchy of agents.

Agent	Freq.	Rationale
Scheduling	Every step (30s)	Jobs arrive continuously
Consolidation	Every 2 steps (1m)	Trend observation
ScaleOut	Every 10 steps (5m)	VM provisioning

D. Reward Functions

Each agent receives a tailored reward signal.

ScaleOut:

$$\begin{aligned} r_{\text{so}} = &-\alpha_1 \cdot \text{underProv} - \alpha_2 \cdot \text{overProv} \\ &- \alpha_3 \cdot \text{runCost} - \alpha_4 \cdot \text{bootPenalty} \\ &+ \alpha_5 \cdot \text{proactiveBonus} \end{aligned} \quad (3)$$

Consolidation:

$$\begin{aligned} r_{\text{con}} = &-\beta_1 \cdot \text{runCost} + \beta_2 \cdot \text{slaBonus} \\ &- \beta_3 \cdot \text{migrationPenalty} - \beta_4 \cdot \text{memPressure} \end{aligned} \quad (4)$$

Scheduling:

$$r_{\text{sch}} = -\gamma_1 \cdot \text{meanWaitTime} - \gamma_2 \cdot \text{slaViolation} \quad (5)$$

The reward coefficients are listed in Table II.

TABLE II: Reward function hyperparameters.

Agent	Parameter	Value
ScaleOut	α_1 (under-prov.)	2.0
	α_2 (over-prov.)	1.0
	α_3 (running cost)	0.5
	α_4 (boot penalty)	0.3
	α_5 (proactive bonus)	0.5
Consolidation	β_1 (running cost)	1.0
	β_2 (SLA bonus)	2.0
	β_3 (migration pen.)	0.5
	β_4 (memory press.)	1.0
Scheduling	γ_1 (mean wait)	1.0
	γ_2 (SLA violation)	2.0

E. Workload Forecaster

The forecaster is a Transformer encoder [2] with MC Dropout [3]:

- **Input:** Last 20 time steps, each with 4 features (CPU utilization, queue length, hour-of-day, day-of-week).
- **Architecture:** 2-layer Transformer encoder, 4 attention heads, 64-dimensional hidden state.
- **Output:** Predictions at 4 horizons ($t{+}1, t{+}5, t{+}10, t{+}15$) at 3 quantiles (10th, 50th, 90th percentile).
- **Uncertainty:** 30 stochastic forward passes with dropout active. Variance across passes $\rightarrow$ uncertainty estimate σ.

F. Safety Coordinator

A rule-based post-processing filter enforces seven hard constraints (Table III).

TABLE III: Safety Coordinator filters.

#	Filter	Rule
1	Boot Protection	Never drain a node younger than boot_time + 60s
2	$N_{\min}$ Floor	Never go below 3 active nodes
3	Uncert. Suppress	If $\sigma_{t+5} > 0.3$, block drains
3b	High-Load Guard	Suppress drains if mean CPU $>$ 75% OR queue $>$ 10 OR any node booting
4	Scale-Out Suppress	Block scale-up if $\geq$2 nodes booting
5	Proactive Scale-Out	If $\sigma > 0.3$ AND CPU rising, force +1
6	Queue Emergency	Force +1 VM if queue > 40; force +2 if queue > 120

The Safety Coordinator guarantees that the RL agents cannot crash the system, even with a poorly trained policy.

V. Implementation

A. Cloud Simulator

We built a discrete-event cloud simulator using **SimPy** wrapped in a **Gymnasium** environment interface. Key parameters:

- **Cluster:** 3–20 VMs with 4 node types (varying CPU/memory).
- **Time step:** 30 seconds. Episode length: 120 steps (1 hour).
- **Node lifecycle:** BOOTING (30–90s) $\rightarrow$ ACTIVE $\rightarrow$ DRAINING (30s grace) $\rightarrow$ TERMINATED.
- **Workload:** Real arrival rates from the Alibaba Cluster Trace 2018 [7], binned at 30-second intervals (2,880 snapshots/-day, 7 days total, 4,023 machines).

A key implementation detail: the `queue_len` metric is captured at its peak *before* the SimPy dispatcher clears the queue each step (`step_peak_queue`), ensuring accurate queue depth reporting.

The simulator produces a 215-dimensional observation at each step: 120 node features (20 nodes $\times$ 6 features), 80 job features (20 jobs $\times$ 4 features), and 15 global features (active node count, queue length, mean CPU/memory, forecaster outputs).

B. Agent Architecture

Each agent uses a 3-layer MLP with LayerNorm:

```
Input(215) -> Linear(512) -> LN -> ReLU
        -> Linear(256) -> LN -> ReLU
        -> Linear(128) -> LN -> ReLU
        -> Output (action logits)
```

The Scheduling Agent uses weight-tied job scoring: each of the top-K jobs is processed through the same network, producing per-job priority scores.

C. Training Pipeline

Training proceeds in two phases:

Phase 1—Forecaster: The Transformer is trained on Day 1 of the Alibaba trace using MSE + quantile loss. Training: 50 epochs with early stopping (patience = 10), Adam optimizer (lr = 10^{-3}).

Phase 2—RL Agents (I-PPO): The three agents are trained simultaneously on the simulator with the fixed forecaster:

- Training steps: 100,000+
- Buffer size: 2,048 steps per update
- PPO epochs per update: 4
- Learning rate: 3×10^{-4} (Adam)
- Entropy coefficient: 0.01 $\rightarrow$ 0.001 (linear decay)
- EMA reward normalization (window = 1,000 steps)

Training was performed on Kaggle (T4 GPU). Forecaster training takes $\sim$20 minutes; RL training takes $\sim$30 minutes.

D. Software Stack

TABLE IV: Technology stack.

Component	Technology
Simulator	SimPy 4.0 + Gymnasium 0.29
RL agents	PyTorch 2.0 (custom PPO)
Forecaster	PyTorch Transformer + MC Dropout
Language	Python 3.11
Training	Kaggle (NVIDIA T4 GPU)
Workload	Alibaba Cluster Trace 2018

E. Interactive Demo

The interactive demo (`demo.py`) supports four modes:

- **Standard**: step-by-step walkthrough of AutoCloud-Agent vs. one chosen baseline.
- **Shootout**: AutoCloud-Agent vs. all five baselines with a ranked results table.
- **Stress**: AutoCloud-Agent vs. Kubernetes HPA on an extreme workload scenario (ramp-up, shock, choppy plateau, or trough + recovery).
- **Ablation**: 3-agent I-PPO vs. Single-Agent PPO, illustrating the value of decomposition.

VI. Evaluation

A. Metrics

B. Baselines

We compare against six baselines spanning industry practice, control theory, and RL (Table VI).

TABLE V: Evaluation metrics.

Metric	Definition
SLA Rate	Fraction of steps where P95 latency < 500ms
Cost Efficiency	$1 - \frac{\text{actual cost}}{\text{max possible cost}}$
CPU Utilization	Mean CPU usage across active nodes
Node Stability	$1 - \frac{\text{std}(N)}{\text{mean}(N)}$

TABLE VI: Baseline methods.

Category	Method	Description
Industry	KubernetesHPA	k8s HPA with 10% dead-band
Industry	AWSTargetTracking	AWS asymmetric cooldowns
Control	MPCController	5-step MPC with EWMA [4]
Rule	ThresholdReactive	CPU >80%: add; <30%: drain
RL ablation	SingleAgentPPO	One agent, all actions
Lower bound	StaticN	Fixed 10 nodes

C. Results

All methods are evaluated on the Alibaba Cluster Trace 2018 (5 episodes $\times$ 3 random seeds). Results are shown in Table VII.

TABLE VII: Evaluation results on Alibaba Cluster Trace 2018.

Method	SLA	Cost	CPU	Stab.
AutoCloud	**100%**	**.962**	**.552**	.889
MPC	100%	.962	.552	**.941**
Threshold	100%	.955	.483	.822
StaticN	100%	.938	.333	1.00
K8s HPA	100%	.930	.312	.842
AWS TT	100%	.928	.295	.876
SinglePPO	100%	.924	.413	.794

D. Analysis

Cost efficiency: AutoCloud-Agent matches MPC (0.962), the best classical method. It outperforms Kubernetes HPA by 3.4% and AWS Target Tracking by 3.7%.

CPU utilization: AutoCloud-Agent achieves 55.2%, nearly double that of AWS Target Tracking (29.5%). Higher utilization means fewer idle resources—directly translating to lower cost.

Multi-agent vs. single-agent: SingleAgentPPO (0.924 cost efficiency, 41.3% CPU) underperforms AutoCloud-Agent (0.962, 55.2%) across all metrics. This confirms the value of multi-agent decomposition and temporal hierarchy.

Stability trade-off: MPC achieves higher stability (0.941 vs. 0.889) because its control-theoretic approach produces naturally smooth trajectories. The RL agent trades some stability for higher responsiveness to demand changes.

SLA compliance: All methods achieve 100% SLA on this trace. The key differentiator is cost—how cheaply they maintain SLA compliance.

E. Ablation: Why Multi-Agent?

The SingleAgentPPO baseline serves as an ablation study. A single agent must control a joint action space of $3 \times 2^{20} \times 5 \approx 15$ million combinations. The I-PPO decomposition reduces this to three independent spaces (3, 2^{20}, 5), enabling:

- Faster convergence (smaller per-agent action spaces)
- Agent-specific reward shaping (clearer learning signals)
- Temporal decomposition (matching decision freq. to task timescale)
- Easier credit assignment for debugging

F. Stress Testing

We evaluate robustness under four out-of-distribution workload patterns:

1) **Ramp-Up:** Linear increase 20% $\rightarrow$ 100% over 120 steps. Tests continuous provisioning.
2) **Sudden Spike:** 30% $\rightarrow$ 95% in 5 steps. Tests reactive and proactive scaling.
3) **Choppy Plateau:** Sustained 70–90% with noise. Tests stability under persistent high load.
4) **Trough-then-Recovery:** Drop to 10%, then recover to 80%. Tests aggressive consolidation and rapid recovery.

The Safety Coordinator's proactive scale-out filter (Filter 5) and queue backlog emergency filter (Filter 6) are critical for the sudden spike scenario, forcing immediate node addition when uncertainty is high, CPU is rising, or the queue exceeds 40 pending jobs.

VII. Key Design Decisions

1) **Gymnasium API:** Wrapping the SimPy engine in Gymnasium enables compatibility with any RL library and fair baseline comparison.
2) **Safety as a separate module:** Hard constraints are enforced in a post-processing filter rather than the reward function. This guarantees safety regardless of policy quality.
3) **MC Dropout over ensemble:** 30 forward passes through one model vs. training 5+ ensemble members. Comparable uncertainty at $5\times$ lower compute.
4) **EMA reward normalization:** Stabilizes training across episodes with varying reward magnitudes.
5) **Real trace data:** Alibaba 2018 (4,023 machines, 7 days) ensures evaluation reflects actual cloud workload patterns.

VIII. Limitations and Future Work

Limitations: Homogeneous VMs (real clouds have heterogeneous instance types); single-cluster scope (production systems manage thousands of clusters); simplified networking (no cross-rack or bandwidth modeling); static reward weights (adaptive shaping could improve performance); single-day training data (multi-day training with domain randomization would improve generalization).

Future Work: Heterogeneous VMs with bin-packing strategies; multi-cluster federation with a hierarchical agent; vertical scaling (resizing individual VMs); transfer learning across

different cloud traces; spot instance awareness (preemption risk + pricing).

IX. Conclusion

We presented AutoCloud-Agent, a multi-agent reinforcement learning system for cloud resource management that simultaneously optimizes for elasticity and cost minimization. By decomposing the problem into three independent sub-agents with temporal hierarchy, incorporating uncertainty-aware forecasting, and enforcing hard safety constraints, AutoCloud-Agent achieves state-of-the-art cost efficiency (0.962) and CPU utilization (55.2%) while maintaining 100% SLA compliance on the Alibaba Cluster Trace 2018. The multi-agent I-PPO approach outperforms single-agent RL by 4.1% on cost and 33.7% on CPU utilization, validating the decomposition strategy. The system matches the best classical method (MPC) while offering greater adaptability to non-stationary workloads.

References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[3] Y. Gal and Z. Ghahramani, "Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning," *International Conference on Machine Learning (ICML)*, 2016.

[4] Q. Zhang et al., "AWARE: Automate Workload Autoscaling with Reinforcement Learning in Production Cloud Systems," *USENIX Annual Technical Conference (ATC)*, 2023.

[5] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource Management with Deep Reinforcement Learning," *ACM Workshop on Hot Topics in Networks (HotNets)*, 2016.

[6] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," *International Conference on Learning Representations (ICLR)*, 2016.

[7] Alibaba Group, "Cluster Trace Program," https://github.com/alibaba/clusterdata, 2018.

[8] Kubernetes, "Horizontal Pod Autoscaler," https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/.